

FCFS: First Come, First Served (FIFO)

Implemented rules and examples for Scheduling Studio. These describe the current visualizer, including its deterministic tie and boundary ordering.

Model and terminology

The visualizer uses one CPU, discrete ticks, and CPU-only processes. A process has an arrival time and a service time. Service is the total CPU work required to finish; remaining service is the work still needed. Waiting does not consume service. There is no I/O, blocking, or context-switch cost in the normal UI.

A snapshot at time t shows the state after the scheduling decisions at t and before execution during $[t, t + 1)$. If a process runs during that interval, its remaining service decreases by one. Completion is recorded at the following boundary. Finished processes remain in the metrics table but leave the CPU and ready queue.

1. Arrivals join the back of the ready queue

Processes enter when their arrival time is reached. Earlier arrivals enter first; processes arriving at the same time retain input order. Process IDs are not a scheduling tie-breaker.

Example: Input order C, A, B with all arrivals at zero gives ready order C, A, B before dispatch. C runs first.

2. Dispatch the queue head

Whenever the CPU is free, remove the first ready process and run it. Service length does not affect selection.

Example: If B needs six ticks and C needs one tick, ready order B, C still dispatches B first.

3. Run until completion

FCFS is non-preemptive. Arrivals do not interrupt the running process, even if they need less service. There is no quantum, allotment, demotion, or boost in this policy.

Example: A starts at zero with five service ticks. B arrives at one with one service tick. A continues through tick 4–5; B can start at time 5.

4. Handle completion before arrivals and dispatch

At each boundary:

1. Remove the running process if its remaining service is zero.
2. Append all arrivals at this time, in input order.
3. If the CPU is free, dispatch the ready-queue head; otherwise keep the current process.
4. Record the boundary state, then run the next tick unless all processes have finished.

Previously waiting processes remain ahead of new arrivals. No tick is lost between one process finishing and the next starting.

Example: A finishes at time 3, B is already waiting, and C arrives at 3. B starts at 3; C joins behind B.

5. Idle only when no process is ready

If the CPU is free and the ready queue is empty, record idle ticks until the next arrival. Future arrivals do not enter the ready queue early. Stop after every process has completed.

Example: A arrives at 3 and needs two ticks. Times 0–3 are idle; A runs during 3–5 and finishes at 5.

Worked example

The following independent example uses input order A, B, C:

Process	Arrival	Service
A	0	5
B	1	2
C	1	1

Interval	CPU	Reason
0–5	A	A arrives first and runs to completion.
5–7	B	B and C arrived together; B was earlier in the input.
7–8	C	C is the remaining queue head.

Per-tick CPU sequence: A A A A A B B C. At time 8, all processes are finished.

Process	Response	Waiting	Turnaround
A	0	0	5
B	4	4	6
C	6	6	7

Metrics and display

- **Response time:** first CPU start minus arrival time. It is recorded once and remains unchanged afterward.
- **Waiting time:** total ticks spent ready but not running. Time before arrival is excluded. At completion, waiting equals turnaround minus original service.
- **Turnaround time:** completion time minus arrival time. It remains unavailable until the process finishes.
- **Averages:** waiting includes every listed process, including future processes with zero waiting. Response averages include only processes that have started; turnaround averages include only finished processes. An unavailable value is shown as a dash. Non-integer averages are displayed to one decimal place.
- **Cards and timeline:** `N service left` is remaining CPU work. A timeline cell at `t` identifies the process running during `[t, t + 1)`, or idle. Animation and reverse playback visualize the same recorded schedule and do not change its decisions.

Input limits and recovery

Use 1–50 processes. IDs must be nonempty and unique after trimming whitespace and ignoring case. Arrival times must be nonnegative safe whole numbers; service times must be positive safe whole numbers. The complete workload, including idle gaps, must finish by tick 2,000. Finishing exactly at 2,000 is allowed; finishing at 2,001 is rejected.

The numeric editor floors fractional entries and clamps them to the field minimum. JSON imports and direct engine calls must pass integer validation. Invalid edits replace the visualization with a validation message; an invalid JSON import leaves the current scenario unchanged. Editing a valid scenario or changing the policy restarts playback at time zero.